

c# Programmieren (c) HTL-Leonding

Rational Number

Lehrziele

- OO Basic
- Methoden und Properties

Aufgabenstellung

Gesucht ist eine Klasse, die rationale Zahlen unterstützt. Rationale Zahlen sind Zahlen, die sich durch einen Bruch darstellen lassen: siehe https://de.wikipedia.org/wiki/Rationale_Zahl

Die gesuchte Klasse muss folgende Funktionalitäten aufweisen:

- Der Eigenschaften **Nenner** (**Numerator**) und der **Zähler** (**Denominator**) sind ganze Zahlen (Datentyp `int`). Sie lassen sich **nur** über den Konstruktor setzen. (Damit ist eine Instanz dieser Klasse unveränderbar = **immutable**)
- **Normalize**
Beim Aufruf dieser Methode wird eine neue rationale Zahl angelegt. Nenner und Zähler werden gekürzt und der Zähler wird positiv.
Beispiele: $3/6$ wird zu $1/2$, $-9/-27$ wird zu $1/3$ und $4/-16$ wird zu $-1/4$.
Hinweis: Gekürzt wird mit dem GGT aus Zähler und Nenner.
- **Add, Sub, Mult, Div**
Alle Methoden verwenden einen Parameter. Die aktuelle Zahl (das aktuelle Objekt) wird mit dieser Zahl addiert, subtrahiert, multipliziert oder dividiert und als neue Zahl zurück gegeben.
- **Inverse**
Berechnet aus z.B. $1/4$ den Wert $-1/4$. Das Ergebnis liefert die Methode als Rückgabewert.
- **Reciprocal**
Berechnung des Kehrwerts, aus z.B $1/4$ den Wert $4/1$.
- **ToString**
Die rationale Zahl wird in einen String konvertiert und als Rückgabewert geliefert. Es gilt:
 - Ergibt ein Bruch 0, liefert die Methode "0"
 - Stellt ein Bruch eine ganze Zahl dar, z.B. $6/3$, wird nur die ganze Zahl ausgegeben. Beispiel $6/3$ wird zu "2"
 - Ist der Zähler 0 (=ungültig, Division durch 0), liefert die Methode "invalid"
 - In allen anderen Fällen wird {Nenner}/{Zähler} ausgegeben (ohne Normalisierung), Beispiel: $22/7$ => "22/7"

Unittest

In dem bereitgestellten Programm-Template sind Unittests vorhanden. Implementieren Sie die Methoden so, dass **alle** Unittests erfolgreich ausgeführt werden können. Änderungen an den *Unittests* sind nicht erlaubt.

Hauptprogramm

Die Klasse wird dazu verwendet, die besten Annäherung von Pi als Bruch zur berechnen. Nenner und Zähler sollen dabei kleiner als 1000 sein.